ðŸ““ ShopSense Viva Guide

Object-Oriented Programming (OOP) & JavaFX - Viva Q&A

Open-Ended Lab Project Defense Preparation

DEVELOPMENT TEAM MEMBERS

Ryan Nasir (Student ID: 74832) - Core OOP Architect

Muhammad Umar (Student ID: 74786) - Backend & Persistence

Muhammad Wahaj (Student ID: 74742) - UI Layout & Styling

Muhammad Asim Abbasi (Student ID: 74844) - Ledger & Analytics

INSTRUCTOR: Faiza Latif Abbasi

SEMESTER: Spring 2026

SLOT: 08:30 to 11:20 - WEDNESDAY

ShopSense: Smart Credit Tracking & Retail Billing System

Object-Oriented Programming (OOP) & JavaFX - Viva Q&A Preparation Guide

This guide compiles critical conceptual and implementation questions that evaluators frequently ask during project defense vivas. The answers are tailored specifically to the design and codebase of the **ShopSense** application.

Part 1: Core Object-Oriented Programming (OOP) Concepts

> **Q1: What is Encapsulation, why is it used, and how is it implemented in ShopSense?**

* **What it is:** Encapsulation is the practice of bundling data (attributes) and the methods that operate on that data into a single unit (a class), while hiding the internal details from outside access.

* **Why it is used:** It protects an object's internal state from unauthorized or accidental modification, ensuring data integrity and validation.

* **How it works & Where applied:** All instance variables in our model classes (like `Person`, `Customer`, `Transaction`, and `Account`) are declared `private`. They can only be accessed or modified via public `get` and `set` methods. For example, `Customer.creditLimit` can only be altered via `setCreditLimit(double limit)` which can include validations (e.g. check for negative values).

> **Q2: What is Inheritance, why is it used, and where is it applied in this project?**

* **What it is:** Inheritance is a mechanism where a new class (subclass/derived class) acquires the attributes and methods of an existing class (superclass/base class).

* **Why it is used:** It promotes code reusability, reduces redundancy, and establishes a logical taxonomy ("is-a" relationship) between entities.

* **How it works & Where applied:**

- `Customer` inherits from `Person` (`public class Customer extends Person`). This means `Customer` automatically inherits `id`, `name`, `phoneNumber`, and `email` without rewriting them.
- `CreditTransaction` and `PaymentTransaction` inherit from `Transaction`.
- `CustomerAccount` inherits from `Account`.

Q3: What is Abstraction, why is it used, and how does it differ from an Interface?

* **What it is:** Abstraction is the process of hiding complex implementation details and showing only the essential features of an object. In Java, it is achieved using abstract classes and interfaces.

* **Why it is used:** It separates the "what it does" (design) from the "how it does it" (implementation), reducing code coupling and making systems easier to maintain.

* **How it works & Where applied:**

- **Abstract Class:** `Account` and `Transaction` are defined as `abstract`. They cannot be instantiated directly (you cannot write `new Transaction(...)`). Instead, they define abstract methods like `getEffectOnBalance()` that subclasses *must* implement.
- **Interface vs. Abstract Class:** An abstract class can have instance fields and concrete helper methods. An interface (like `Alertable`) is a pure contract declaring behaviors without state. A class can extend only one abstract class but can implement multiple interfaces.

Q4: What is the Alertable Interface, why is it used, and how does it work?

* **What it is:** `Alertable` is a custom Java interface containing two abstract methods: `checkCreditLimit()` and `sendAlert()`.

* **Why it is used:** It decouples the alert-checking logic from the general customer structure, creating a contract that any account-based entity can implement to trigger credit limit alerts.

* **How it works & Where applied:** The `Customer` class implements `Alertable` (`implements Alertable`).

- `checkCreditLimit()` returns `true` if the customer's outstanding balance exceeds their credit limit.
- `sendAlert()` throws a custom `CreditLimitExceededException` if the check returns `true`.

> **Q5: What is Polymorphism, and what is the difference between Overloading and Overriding?**

* **What it is:** Polymorphism means "many forms". It allows objects of different classes to be treated as objects of a common superclass.

* **Method Overriding (Runtime Polymorphism):**

- *What:* Redefining a superclass method in a subclass with the same signature.
- *Where:* `CreditTransaction` and `PaymentTransaction` override `getEffectOnBalance()`. When iterating over transactions in `CustomerAccount.calculateBalance()`, calling `t.getEffectOnBalance()` dynamically executes the correct subclass logic (adding for credit, subtracting for payments).

* **Method Overloading (Compile-time Polymorphism):**

- *What:* Defining multiple methods in the same class with the same name but different parameters.
- *Where:* `CustomerAccount` overloads `addTransaction()`. It has three signatures accepting different parameters (e.g., one with a date and category, one without category defaulting to "Others", and one accepting a pre-built `Transaction` object).

Part 2: JavaFX & Desktop UI Architecture

> **Q6: Why do we have a Main class that does not extend Application?**

* **What it is:** Our entry point `com.shopsense.Main` simply has a `main` method calling `Application.launch(ShopSenseApp.class, args)`. It does not inherit from `javafx.application.Application` itself.

* **Why it is used:** In Java 11 and later, if the class containing the `main` method extends `Application`, the Java launcher checks for the JavaFX modules in the default JDK classpath. If they are not found, it throws the error: *"JavaFX runtime components are missing"*. Using a separate launcher class bypasses this check, allowing the standalone SDK to load dynamically via VM arguments.

> **Q7: What is event-driven programming, and how is it used in ShopSense?**

* **What it is:** A programming paradigm where the flow of the program is determined by events such as user clicks, key presses, or sensor outputs.

* **How it works & Where applied:** JavaFX UI elements (like buttons and text fields) register action listeners using Lambda expressions. For example:

`btnSave.setOnAction(e -> { ... save logic ... });`

When the user clicks the save button, the JavaFX application thread intercepts the click event and fires the lambda function.

> **Q8: How did we customize the UI layout without FXML?**

* **What it is:** Programmatic UI construction. Instead of loading an XML file, we instantiate layout container objects (like `VBox`, `HBox`, `GridPane`, and `StackPane`) and controls directly in Java code.

* **Why it is used & Where applied:** Programmatic layout is highly robust, compiles cleanly, avoids XML parsing overhead, and makes it easy to bind dynamic Java variables to labels and tables. We styled this layout by attaching a CSS stylesheet (`scene.getStylesheets().add("styles.css")`) to set colors, borders, and rounded corners.

Part 3: Data Persistence & File I/O

> **Q9: How is data stored, and why did we choose file handling over a database?**

* **What it is:** Data is persisted in raw text format under `data/customers.txt` and `data/transactions.txt`.

* **Why it is used:** File handling is lightweight, requires no database installation or drivers (like MySQL or JDBC), and satisfies the standard Open-Ended Lab academic requirement of using native Java File I/O.

* **How it works:**

- **Saving:** `StorageManager` loops through the customers list, serializing their details into pipe-separated string lines (`id|name|phone|email|limit`) and writes them using a `BufferedWriter`.
- **Loading:** It reads the text files line-by-line using a `BufferedReader` and splits them (`split("\\|")`) to reconstruct the objects.

> **Q10: How does the system link transactions back to customers on startup?**

* **What it is & Why it is needed:** The transactions file doesn't store objects; it stores plain text strings containing a foreign key `customerId`.

* **How it works:** During startup, `StorageManager.loadData()` first reads all customer profiles from `customers.txt` into memory. It then reads the transactions from `transactions.txt`. For each transaction line, it extracts the `customerId` and performs a **linear search loop** through the customers list. When a match is found, it calls `customer.getAccount().addTransaction(t)` to restore the customer's ledger.

Part 4: Exceptions & Exception Handling

> **Q11: What are Custom Exceptions, and why did we create CreditLimitExceededException?**

* **What it is:** A custom exception is a user-defined exception class that extends the base Java `Exception` class.

* **Why it is used:** Standard Java exceptions (like `NullPointerException` or `IOException`) do not represent business rules. Defining `CreditLimitExceededException` allows the program to explicitly catch and handle credit limit breaches separately from system errors.

* **How it works:** When logging a credit transaction, the system calculates the simulated new balance. If it exceeds the customer's limit, the code triggers:

`throw new CreditLimitExceededException(...)`

The GUI catches this specific exception in a `try-catch` block and opens the override confirmation dialog.